

Class Design

Advanced Programming
Brent van Bladel



Classes

A class is user-defined type providing an abstraction to a concept.

- Group data that forms a larger concept together.
- Link functionality to data.

```
1. class Reporter {  
2.     public:  
3.         Reporter();  
4.         ~Reporter() = default;  
5.  
6.         void print(std::string_view msg);  
7.     private:  
8.         std::ostream& _out;  
9. };
```

Classes vs Structs

A class is user-defined type providing an abstraction to a concept.

A struct is user-defined type providing an abstraction to a concept.

```
1. struct Coordinate {  
2.     int x, y; // public  
3. };  
4.  
5. class Coordinate {  
6.     int x, y; // private  
7. };
```

Difference: default access specifier

When to use structs? Rule of thumb: if all members are public, use a struct.

Class Interface

A class interface is the set of data members and member functions that are visible to the outside.

The class interface consists of all public members.

The implementation details are hidden to the user.

```
1. class Reporter {  
2.     public:  
3.         Reporter();  
4.         ~Reporter() = default;  
5.  
6.         void print(std::string_view msg);  
7.     private:  
8.         std::ostream& _out;  
9. };  
10.  
11. Reporter::Reporter() : _out{std::cout} {}  
12.  
13. void Reporter::print(std::string_view msg){  
14.     _out << msg << std::endl;  
15. }
```

Class Interface - ADT

A well-designed class interface should form an **Abstract Data Type**.

The class interface promises a certain set of properties and abilities:

- Pre- & post-conditions
- Invariants

Advantages

- **Encapsulation**
The user does not need to know the implementation details.
- **Localization of change**
Changes to the ADT are hidden behind the interface.

Class Interface - Header files

A class interface is typically defined in a header file.

The class implementation is defined in a source file.

```
1. class Reporter {  
2.     public:  
3.         Reporter();  
4.         ~Reporter() = default;  
5.  
6.         void print(std::string_view msg);  
7.     private:  
8.         std::ostream& _out;  
9. };
```

Header

```
1. Reporter::Reporter() : _out{std::cout} {}  
2.  
3. void Reporter::print(std::string_view msg){  
4.     _out << msg << std::endl;  
5. }
```

Source

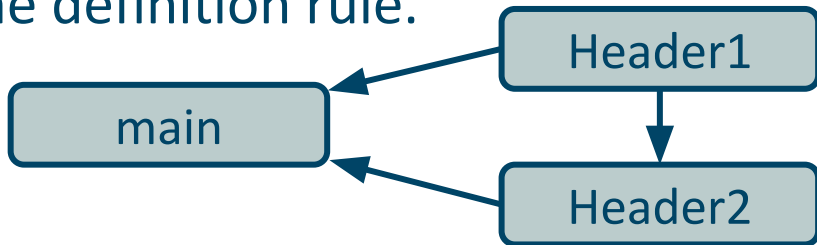
#Include

When including a file, the preprocessor adds that entire file to the translation unit.

Header is compiled multiple times!

Dangers:

- Breaking the one definition rule.



- Cyclic includes.



#Include

Solutions:

- Breaking the one definition rule:
include guards.
- Cyclic includes:
forward declaration.

```
1. #ifndef REPORTER
2. #define REPORTER
3.
4. class Reporter {
5.     // code
6. };
7.
8. #endif
```

```
if REPORTER not
defined
{
    define REPORTER
    //code
}
```

```
1. class Message;
2.
3. class Reporter {
4.     public:
5.         void print(Message msg);
6. };
```

Message

Reporter



Modules (since C++20)

```
1. // main.cpp
2. import helloworld; // import declaration
3.
4. int main() {
5.     hello();
6. }
```

```
1. // helloworld.cpp
2. export module helloworld; // module declaration
3.
4. import <iostream>; // import declaration
5.
6. export void hello() { // export declaration
7.     std::cout << "Hello world!";
8. }
```

Modules allow sharing of declarations and definitions across translation units while only compiling them once!

const member function

A **const member function** specifies that the function does not modify the state of the object.

```
1.  class Reporter {  
2.      public:  
3.          Reporter();  
4.          ~Reporter() = default;  
5.  
6.  
7.          void print(std::string_view msg) const;  
8.      private:  
9.          std::ostream& _out;  
10. };  
11.
```

mutable data member

A mutable data member

specifies that the data can be modified by a const member function.

Why?

Physical const vs Logical const

```
1. class Reporter {
2.     public:
3.         Reporter();
4.         ~Reporter() = default;
5.         void print(std::string_view msg) const;
6.     private:
7.         std::ostream& _out;
8.         mutable std::queue<std::string> _buffer;
9. };
10.
11. void Reporter::print(std::string_view) const {
12.     if (!_out.good())
13.         _buffer.push(msg);
14.     else
15.         _out << msg << std::endl;
16. }
```

Physical const vs Logical const

Physical const: The method is const from a memory perspective. The object in memory does not change.

Example: A “getter” function.

Logical const: The method is const from an interface / ADT perspective. As far as the user knows, the object does not change.

Example: A print function that buffers messages if needed.

Static data members

A **static data member** is a data member that is part of a class, but not part of an object of that class.

There is exactly one copy of a static member in memory instead of one copy per object!

```
1. class Reporter {  
2.     public:  
3.         Reporter();  
4.         ~Reporter() = default;  
5.         void print(std::string_view msg);  
6.     private:  
7.         std::ostream& _out;  
8.         static unsigned int _totalNumberOfMessages;  
9. };
```

Static member functions

A **static member function** is a member function that is part of a class, but not part of an object of that class.

You can call a static member function without an object of the class!

```
1.  class Reporter {
2.      public:
3.          Reporter();
4.          ~Reporter() = default;
5.
6.          static void print(std::string_view msg){
7.              std::cout << msg << std::endl;
8.          }
9.  };
10.
11.  int main() {
12.      Reporter::print("Hello World!"sv);
13.  }
```

Operator Overloading

Customize the C++ operators for operands of user-defined types.

```
1.  class Reporter {  
2.      public:  
3.          Reporter();  
4.          ~Reporter() = default;  
5.  
6.          void operator<<(std::string_view msg){  
7.              std::cout << msg << std::endl;  
8.          }  
9.  };  
10.  
11. int main() {  
12.     Reporter reporter;  
13.     reporter << "Hello World!"sv;  
14. }
```

Operator Overloading

Restrictions

- Some operators cannot be overloaded (sizeof, typeid,...)
- No new rules can be defined (precedence, associativity,...)
- No new tokens or symbols can be defined (\$, @,...)
- Only for user-defined types

Note

The operators = and & and , are synthesized if not explicitly defined